

ROCA Codex Build Guide

Target System: ROCA Capsule Agreement Engine (v1.3 + Minimal Kernel) **Purpose:** Instructions for Codex or autonomous coding agents to build, extend, and maintain the ROCA system.

1. System Overview

ROCA (Routed Orbital Capsule Architecture) is a deterministic, capsule-based cognitive engine built around:

- Semantic Capsules (core data units)
- Routing-by-Agreement (consensus computation)
- Orbital hierarchy (0 = core cognition, 3 = raw input)
- Gravity-based relevance weighting
- Temporal graph memory (frame-based persistence)
- Hardware-aware adaptive execution (Resource Manager)

The system is NOT a neural network. It is a structured symbolic runtime.

2. Core Build Goal

Codex must produce a working system that:

1. Boots a ROCA kernel (`roca_minimal_kernel.py`)
 2. Initializes capsule graph with seed knowledge
 3. Runs deterministic routing-by-agreement loops
 4. Updates gravity + orbit levels per iteration
 5. Logs temporal frames for replay
 6. Respects hardware-aware execution budgets
-

3. Required File Structure

```
roca/
├── core/
│   ├── capsule.py           # Capsule definition + merge logic
│   ├── routing.py          # Agreement + routing engine
│   ├── orbit.py            # Gravity + orbit transitions
│   ├── timeline.py         # FrameNode + temporal graph
│   └── adaptive_resource.py # Hardware-aware budget manager
└── engine/
```

```

├── kernel.py          # ROCAKernel main loop
├── archive/
│   ├── persistent_store.py    # Capsule persistence layer
│   └── runtime/
│       ├── bootstrap.py      # System startup entry
│       └── data/
│           ├── initial_capsules.json # Seed capsule graph
│           └── main.py        # Entry point

```

4. Execution Model

Codex must implement a **deterministic loop engine**:

Main Loop

```

while system_running:
    ingest_input()
    active_set = select_capsules_by_budget()
    active_set = route_by_agreement(active_set)
    update_gravity(active_set)
    update_orbits(active_set)
    write_temporal_frame()
    persist_state()
    hibernate_low_gravity_capsules()

```

5. Capsule Specification

Each capsule MUST implement:

```

Capsule:
    id: str
    type: str
    state: dict
    confidence: float
    timestamp: float
    orbit_level: int
    gravity: float

```

```
lane: str
hibernating: bool
```

Required Methods

- `evaluate_agreement(other_capsule) -> float`
- `merge(other_capsule)`

Merge must be deterministic except for controlled stochastic blending in state resolution.

6. Routing-by-Agreement Rules

Codex must implement:

- Pairwise capsule comparison
- Threshold-based merging (default: 0.63)
- Fixed routing rounds (default: 4)

Rule:

```
if A(Ci, Cj) > 0.63:
    merge(Ci, Cj)
```

No backpropagation or gradient updates allowed.

7. Gravity System

Gravity defines activation strength:

- Increases after successful agreement
- Decays implicitly via inactivity
- Controls orbit transitions

Rule:

```
g = min(1.0, g + delta)
```

8. Orbit System

Orbit	Meaning
0	Core cognition (stable identity)
1	Active reasoning
2	Working memory
3	Raw ingestion

Transition rule:

```
if gravity > 0.8:  
    orbit_level -= 1
```

9. Adaptive Resource Manager

Codex must implement hardware-aware constraints:

Responsibilities

- Detect CPU / RAM
- Set max active capsule budget (B)
- Dynamically reduce workload under pressure

Behavior

- High load → reduce active set size
- Low load → expand active cognition window

No external cloud scaling assumed.

10. Temporal Graph System

Every loop iteration produces a frame:

```
FrameNode:  
    timestamp  
    active_capsules
```

```
orbit_distribution
edges (agreement relations)
```

All frames are appended to a persistent timeline.

This enables full replay of system cognition.

11. Boot Sequence

Codex must implement:

1. Kernel initialization
 2. Seed capsule injection
 3. Resource manager activation
 4. Main loop execution
 5. Timeline logging
-

12. Determinism Rules

ROCA must remain:

- Deterministic across runs (same seed → same structure)
- Free of gradient descent
- Free of stochastic training loops

Randomness is allowed ONLY in controlled merge blending.

13. Output Expectations

System output must include:

- Active capsule count
 - Orbit distribution
 - Max gravity value
 - Frame count
-

14. Design Philosophy

ROCA is not an LLM.

It is:

A structured cognitive graph that evolves through agreement, not prediction.

15. Build Priority Order (Codex Guidance)

1. capsule.py
 2. routing.py
 3. orbit.py
 4. adaptive_resource.py
 5. kernel.py
 6. timeline.py
 7. bootstrap.py
 8. main.py
-

16. Final Constraint

If a design decision introduces neural networks, training loops, or gradient descent:

➡ Reject it immediately ➡ Replace with capsule agreement logic

End of Build Guide